Fair Benchmarking Design Plan: select, poll, epoll and io_uring

Draft v3, October 30, 2026. Written for the whole team: you do not need to have followed any earlier discussion. Items marked **[SETUP]** depend on the final hardware and network setup and are collected in section 14. Items marked **[CONFIRM]** need a team decision.

1. Read this first

We are building three servers (echo, file transfer, Apache web) that can each wait for network events in four ways (select, poll, epoll, io_uring), and benchmarking them so that any difference in the numbers can be blamed on the mechanism alone.

Everyone reads sections 1 to 6 and 11 to 15, then their own server section. The deliverables are: working servers, correctness test reports, benchmark results (medians of repeated runs), and a report that explains why the numbers look the way they do (why select and poll slow down with many connections, how many system calls each mechanism makes per message, what io_uring costs and gains).

Role	Read	You deliver	Owner
Everyone	Sections 1 to 6, 11 to 15	Understand the rules; review each other's code	
Echo server owner	Section 7	apps/echo.c, echo tests, echo results	
File-transfer server owner	Section 8	apps/filesrv.c, file tests, file results	
Apache / web server owner	Section 9	Patched Apache builds, web tests, web results	

Role	Read	You deliver	Owner
I/O module owners	Section 10	io/io_select.c, io_poll.c, io_epoll.c, io_uring.c	

Order of work: (1) shared core and the four I/O modules, (2) the three servers on the baseline, (3) correctness tests, (4) a pilot run to fix parameters, (5) baseline experiments, (6) extensions switched on one flag at a time, (7) analysis and report.

2. Glossary

Term	Plain meaning	Why it matters here
Connection (socket)	One open TCP link between a client and the server; the server holds many at once	Every test varies how many are open
I/O multiplexing	One thread watching many connections and handling whichever needs attention	The four mechanisms are four ways to do this
select	Hands the kernel the whole list of connections on every call and asks which are ready; maximum 1,024	Cost grows with the list size; hard limit
poll	Same idea as select without the 1,024 limit	Still scans the whole list on every call
epoll	The kernel remembers the list and reports only the ready ones	Cost grows with active, not total, connections
io_uring	You queue requests (read this, write that) in a shared ring and the kernel reports completions	Fewer system calls; a different programming model
Readiness vs completion	Readiness: the kernel says a socket is ready, then you do the read. Completion: you ask for the read and the kernel says when it is done	The main split between the first three and io_uring

Term	Plain meaning	Why it matters here
System call (syscall)	A request from your program to the kernel; each has a fixed cost	We count syscalls per message
Byte stream and framing	TCP does not keep message boundaries: two sends can arrive as one read	Every server must find message ends itself
Partial read or write	A call that handled only some of the bytes	Must be handled the same way everywhere
Non-blocking	Calls return at once instead of waiting; EAGAIN means nothing available now	Required for a single-threaded loop
Backpressure	A slow client stops reading, so replies pile up	The server must pause reading that client, never buffer without limit
Delay vs no delay	Delay: Nagle's algorithm is on (kernel default), so small sends may be held briefly and bundled. No delay: TCP_NODELAY is set and data goes out at once	TCP_NODELAY is globally enforced
Baseline	The most basic version of each mechanism, hardcoded	The fair starting point
Extension (flag)	A tuned feature switched on from the command line, one at a time	Lets us say which feature caused a gain
Load generator	An existing program that opens connections and sends traffic to our server	We use existing tools, not our own
Closed loop	Each client waits for the reply before sending again	Finds maximum throughput
Open loop	Clients send on a fixed schedule whatever replies arrive	Shows latency at a chosen load

Term	Plain meaning	Why it matters here
Throughput	Completed requests (or MB) per second	Headline number
Latency, p50, p99	Time for one request; p50 = typical, p99 = the slowest 1%	Averages hide the slow requests
Warm-up, repeat, median	A discarded start of each run; repeated runs; the middle result	How we get stable numbers
Loopback	Client and server on one machine, talking through the kernel	Optional; removes any cable limit
Page cache	RAM the OS uses to keep recently read files	File and web tests read from it, not from disk
sendfile	The kernel copies a file to a socket directly	A file and web extension
MPM, APR pollset	Apache's process and thread model; Apache's wrapper over select, poll and epoll	Where we modify Apache

3. What fair means

Fair means every mechanism is used the way it normally is, with the same application code, the same workload and the same measurement method, and that the only change between two runs of the same mechanism is one flag. We never modify, handicap or tune a mechanism to look like another. Differences that come with a mechanism's own design (io_uring needs a buffer for every pending read, select scans its whole set on every call) are results to report, not defects to remove. The diagram shows the ten decisions and why each exists; the rest of this plan spells them out.



Read it top to bottom: each card names a decision, what we chose, and why that keeps the comparison fair.

The eight fairness rules, which every decision below serves:

- **P1 Same behavior on the wire.** Given the same input bytes, every server variant returns the same output bytes and handles errors the same way. The correctness gate (section 13) enforces it.
- **P2 Mechanisms are used as they exist.** We never modify, handicap or tune a mechanism to resemble another. The baseline is the plain, normal way to use each one. Differences that come with a mechanism's own design (io_uring needs a buffer for every pending read, select scans its whole set on every call) are results to report, not defects to remove.
- **P3 One flag at a time.** Only the baseline is hardcoded. Every extension is a command-line flag, off by default, and two runs of the same mechanism differ by one flag.
- **P4 Correct before fast.** A server that fails the correctness gate produces no accepted results.
- **P5 The client and the network are not the bottleneck.** If the wire saturates, say so and compare CPU cost per message instead of throughput. **[SETUP]**
- **P6 Measure without disturbing.** Read counters before and after a run, never during, and use the difference. Heavy profiling runs separately.
- **P7 Honest statistics.** Repeat runs, report the median and the spread, and test differences instead of eyeballing them.
- **P8 Report differences, do not hide them.** Inherent differences and anything we cannot control are written down (section 15).

4. Code structure

Each server's logic lives in one file. The four I/O mechanisms live in four separate module files that any server plugs in. Echo and file transfer each get one app file. The third server is Apache, which is existing code: it has no app file, but the same four mechanisms go into it (section 9).

- core/ connection table, stats counters, flag parser, logging (shared by all)
- io/ io_select.c io_poll.c io_epoll.c io_uring.c
- apps/ echo.c filesrv.c
- apache/ patches and build scripts
- tools/ run scripts, result parsers, plot scripts
- tests/ correctness gate

results/ one folder per run

That is 2 app files and 4 module files, and they combine into 12 builds:

Server	Builds	Comes from
Echo	echo+select, echo+poll, echo+epoll, echo+uring	apps/echo.c + one io/ module
File transfer	file+select, file+poll, file+epoll, file+uring	apps/filesrv.c + one io/ module
Apache	apache+select, apache+poll, apache+epoll, apache+uring	Apache source + patches in apache/

The plug-in contract between an app and a module is a set of I/O requests, not readiness events. The app says what it wants done and the module says when it is done. The module decides how: a readiness module waits until a socket is ready and then makes the system call, while io_uring queues the request. The same app file therefore works with all four modules, and no module is bent into another's shape.

// the app calls the module

- io_init(config) parse flags, create the epoll fd or ring, etc.
- io_listen(port)
- io_run() the event loop; returns at shutdown
- io_read(conn, max) ask for up to max bytes
- io_get_buf(conn, &cap) a buffer the app can fill before io_write
- io_write(conn, buf, len) send len bytes; buf stays unchanged until on_write_done
- io_read_file(conn, buf, len, fd, off) read a file range into buf (baseline: pread)
- io_send_file(conn, fd, off, len) kernel-side file send; only with --sendfile or --uring-splice
- io_close(conn)
-
- // the module calls the app, always from inside io_run
- on_accept(conn)
- on_read_done(conn, buf, n) n > 0 data, n == 0 peer closed
- on_write_done(conn) every byte of the last io_write was accepted by the kernel
- on_file_read_done(conn, buf, n)

on_close(conn, reason)

Rules for the contract:

- The app never includes epoll.h, poll.h or liburing.h. If an app needs a mechanism-specific line, the interface is missing something: fix the interface and tell everyone.
- **Callbacks run only from inside io_run, never from inside an io_ call.** A readiness module whose request finishes at once (for example a send that succeeds immediately) puts the completion on an internal list that io_run delivers before it waits again, using a zero timeout while the list is not empty. This stops unbounded recursion (on_read_done calling io_write calling on_write_done calling io_read, and so on) and is the same in all four modules.
- At most one read or write request per connection is in flight. The state machine is ACCEPTED, READING, WRITING, back to READING, or CLOSING. Unsent output per connection is therefore never more than one write, and a slow client is paused automatically (backpressure).
- **Buffers belong to the module, because each mechanism has its own natural arrangement** (section 5.3). Data passed to on_read_done is valid only until the callback returns. Anything the app needs to keep must be copied out.
- **Carry slot.** Bytes the app cannot use yet, such as a partial frame or the start of the next pipelined request, go into a small per-connection carry slot taken from a preallocated pool only when needed. The app treats carry plus new data as one stream. This is shared app code, identical in all builds, and it is what prevents pipelined requests from being overwritten when the read buffer is reused.
- The buffer passed to io_write must stay unchanged until on_write_done. Readiness modules copy any unsent remainder into their own on-demand output slot, so they never rely on it after io_write returns.
- The mechanism is chosen at compile time (for example `make APP=echo IO=epoll`), so no function-pointer cost is added to the hot path.
- Every build uses the same compiler flags: C17, gcc -O2 -g -fno-omit-frame-pointer. Record the gcc and liburing versions.
- Servers are single-threaded and pinned to one core. Apache is the exception (section 9).
- Changes to core/ or to the contract need review by every owner, because they affect all builds.

Baseline modules: each mechanism as it is normally used, nothing else.

Module	Baseline
io_select	POSIX select; read and write sets rebuilt every loop; one shared scratch buffer for reads; descriptors at or above 1,024 refused
io_poll	poll over a compact pollfd array; one shared scratch buffer for reads

Module	Baseline
io_epoll	Level-triggered epoll; each socket added once with EPOLLIN; reads into one shared scratch buffer; sends are attempted at once with send(), and EPOLLOUT is enabled only when a send comes back incomplete; epoll_wait with maxevents 256
io_uring	liburing; plain ACCEPT, RECV and SEND; one buffer per connection, because RECV must be given a buffer before data arrives; one submit per loop iteration; ring sized for the run (section 10.5)

Everything beyond this is a flag (section 6). A flag changes exactly one thing; an invalid combination (for example multishot receive without provided buffers) is refused at startup with a clear error; and every server prints its full flag set at startup and in its stats output.

5. Rules every server follows

These rules are the same in every build. They keep the application behavior and the workload identical while each mechanism is used the way it normally is.

5.1 Sockets, accepting, errors

- Sockets are non-blocking (SOCK_NONBLOCK) and close-on-exec (SOCK_CLOEXEC). The listener uses SO_REUSEADDR and a backlog of 4096, so net.core.somaxconn must be raised to match. **[SETUP]** sysctl values on both machines.
- SO_RCVBUF and SO_SNDBUF stay at kernel defaults and are recorded. Flags can change them (section 6).
- **Accepting:** select, poll and epoll accept in a loop until the queue is empty (EAGAIN). io_uring keeps one ACCEPT request armed and re-arms it after each completion. This is how each mechanism is normally used. `--accept-batch N` changes it: readiness modules accept at most N per loop iteration, and io_uring keeps N ACCEPT requests outstanding. `--uring-multishot-accept` is the io_uring feature that lifts the one-at-a-time limit.
- Before a measured invocation starts, the orchestrator waits until the server's accept counter shows that every connection has been accepted, so a slow ramp never counts as measured time.
- The connection limit is configurable. Connections beyond it are closed at once; select refuses any descriptor at or above 1,024.
- **Errors are split in two.** `peer_resets` counts ECONNRESET and EPIPE. These are normal when a load generator exits with unread data in its socket, because the kernel then sends a reset, so they are informational. `errors` counts everything else unexpected (bad framing, failed system calls) and must be zero. SIGPIPE is ignored and EINTR is retried. Short reads are normal on a TCP stream and are never errors.

- Partial writes: the module keeps the unsent bytes and finishes the write before any further read on that connection.
- No idle timeouts in the baseline.

5.2 Delay versus no delay

Nagle's algorithm is the kernel's habit of holding a small send while earlier data is still unacknowledged, so that small sends can be bundled. We call it **delay** (the kernel default) and compare it with **no delay** (TCP_NODELAY set). This is not network delay, so do not confuse it with adding latency using tc netem.

- **TCP Setting Standard**: All test environments operate exclusively in **no delay** mode (`TCP_NODELAY` enabled). Nagle's algorithm (kernel buffering of small sends while unacknowledged data is in flight) is disabled across all components. This controls socket-level packet framing and is separate from network latency introduced via `tc netem`.
- **Server Default**: `--nodelay` is **on by default**. `TCP_NODELAY` is set on every accepted socket upon connection setup.
- **Load Generator Alignment**: Benchmarking tools like `wrk` and `wrk2` enable `TCP_NODELAY` by default. Enforcing no delay on the server ensures out-of-the-box parity with client generators without requiring custom tool patches, command-line modifications, or `LD_PRELOAD setsockopt` shims. If using `tcpkali`, ensure its `--nodelay` option is enabled.
- **Simplified Testing Scope**: Separate delay vs. no-delay comparison runs (formerly E9) and dual-run execution overhead (section 12.7) are eliminated. All benchmarks (file, web, and RPC) run strictly under the universal no-delay configuration, reducing variable matrix size.
- **Send Call Strategy**: Operating with `TCP_NODELAY` eliminates Nagle-induced ACK delays on sequential small writes. However, bundling headers and payloads into a single send operation (via unified buffers, `writev`, `MSG_MORE`, or `sendfile`) remains standard practice to reduce system call overhead and TCP header duplication.
- **Apache Integration**: Apache sets `TCP_NODELAY` by default. Because no delay is the standard across all runs, Apache requires no patches or custom configuration changes.

5.3 Memory: each mechanism uses its own natural buffers

We do not reshape one mechanism to match another. The readiness mechanisms and io_uring need different things, and that difference is part of what the benchmark measures.

Mechanism	Buffers in the baseline
select, poll, epoll	One shared scratch buffer for all reads, because they read only after a socket is ready. Per connection there is only a small state record. If a send comes back

Mechanism	Buffers in the baseline
	incomplete, the module copies the unsent bytes into an output slot taken on demand from a pool.
io_uring	One buffer per connection, because the kernel must be given a buffer before data arrives. The flag `--uring-provided-buffers` replaces these with a shared pool that the kernel hands out only when data arrives.

Consequences, to be reported and not hidden: with thousands of connections the io_uring baseline touches much more memory (for example 10,000 connections times 4 KB is 40 MB), which shows up in peak memory and cache-miss counts (E1, E2). The provided-buffers extension then shows how much of that cost it buys back.

The carry slots and output slots come from a pool mapped once at startup, so there is no malloc or free inside the event loop. Pages are touched, and so counted in memory use, only when a slot is actually used.

Server	`--buf-size` (read size; scratch buffer, or per-connection buffer for io_uring)
Echo	4 KB
File transfer	64 KB
Apache	Apache's own buffers; record what it uses per connection **[CONFIRM]**

Sizes are starting values and are frozen after the pilot (section 12.5). Every mechanism in one server uses the same `--buf-size`, so each read asks for the same number of bytes.

5.4 Message and framing sizes

TCP delivers a stream of bytes, so two 512-byte sends can arrive as one 1,024-byte read or as 300 plus 724 bytes. Each server must find message boundaries itself, and the rule is fixed per server:

Server	What counts as one message	Default	Values to test	Flag
Echo	A fixed-size frame; the server echoes only complete frames	512 B	64 B, 512 B, 4 KB, 16 KB	`--msg-size`
File transfer	Request: one HTTP-style GET line plus a blank line (max 4 KB). Response: status line, Content-Length, blank line, then the file bytes in chunks	64 KB chunks	File sizes 4 KB, 64 KB, 1 MB, 100 MB	`--chunk-size`
Apache	Standard HTTP/1.1 GET; response size = file size	Files of 1 KB, 64 KB, 1 MB	/small, /medium, /large	none

The echo server reads up to `--buf-size` bytes at a time and echoes every complete frame it finds, as a real stream server does. It counts frames per read, so TCP bundling is visible in the results and never hidden. The client's message size must equal `--msg-size` in every echo run, or the server will mis-frame the stream.

Messages per second at the wire limit for a 1 Gbps link (approximate, per direction): 64 B about 810,000; 512 B about 207,000; 4 KB about 29,000; 16 KB about 7,000. **[SETUP]** These move with the real link speed; recompute once the setup is final. If the wire fills first, compare CPU per message and latency instead of throughput (P5).

6. Feature flags

Only the baseline is hardcoded. Every other feature is a command-line flag that is off by default and changes exactly one thing. Flags are parsed in core/ and passed to the module and the app, so any server accepts any flag that applies to it and refuses the rest. Section 6.6 says which experiment tests each flag, so no flag is built without a purpose.

6.1 Common flags (all servers)

Flag	Default	What it changes
`--nodelay`	On (no delay)	Sets TCP_NODELAY on every accepted socket

Flag	Default	What it changes
`--port N`	set per run	Listening port
`--max-conns N`	set per run	Connection limit; extra connections are closed at once
`--buf-size N`	4 KB echo, 64 KB file	Read size: the shared scratch buffer for select, poll and epoll, the per-connection buffer for io_uring
`--backlog N`	4096	listen() backlog
`--rcvbuf N`, `--sndbuf N`	kernel default	SO_RCVBUF and SO_SNDBUF
`--accept-batch N`	Module's normal behavior (readiness: drain until EAGAIN; io_uring: 1 outstanding)	Readiness modules: maximum accepts per loop iteration. io_uring: number of ACCEPT requests kept outstanding
`--busy-poll USEC`	off	SO_BUSY_POLL; only works if the NIC and driver support it **[SETUP]**
`--cpu N`	none	Pins the server thread to core N **[SETUP]**
`--stats-file PATH`	stdout	Writes counters as JSON on SIGUSR1 and at exit
`--msg-size N`	512	Echo frame size
`--chunk-size N`	64 KB	File-transfer write chunk size

6.2 select and poll

No feature flags. These mechanisms have no tuning features, and select's 1,024 limit is a result to report, not a problem to hack around.

6.3 epoll flags

Flag	Baseline	On	Expected effect
`--epoll-et`	Level-triggered	Edge-triggered. The module must read and write until EAGAIN, so it keeps a per-connection flag for connections whose read filled the buffer without reaching EAGAIN and revisits them before waiting again (with a zero timeout), because the kernel will not announce data it has already announced	Fewer wakeups for busy connections
`--epoll-maxevents N`	256	Events fetched per epoll_wait	Batch size versus per-call cost
`--epoll-oneshot`	off	EPOLLONESHOT; re-arm after each event	Shows the cost of re-arming

6.4 io_uring flags

Flag	Baseline	On	Needs
`--uring-entries N`, `--uring-cq-entries N`	Submission ring sized per run to at least the connection count (next power of two); completion ring left at liburing's default of twice that	Explicit ring sizes	none
`--uring-submit-batch N`	Submit once per loop iteration	N = 1 submits after every request; larger N submits after N queued	none

Flag	Baseline	On	Needs
`--uring-fixed-files`	Normal descriptors	Registered file table and direct accept; saves a descriptor lookup per operation	none
`--uring-provided-buffers`	One buffer per connection	Shared provided buffer ring; buffers are handed out only when data arrives and returned after the data is used or sent	none
`--uring-multishot-accept`	One ACCEPT re-armed each time	One request keeps producing connections	none
`--uring-multishot-recv`	One RECV per read	One request keeps producing data, possibly several frames or part of one per completion	provided buffers
`--uring-fixed-buffers`	Normal buffers	Registered (pinned) buffers; register one contiguous arena as a single entry, because the kernel accepts at most 1,024 registered buffers per registration; pinned memory is charged to the locked-memory limit (`ulimit -l`) **[SETUP]**	none
`--uring-single-issuer`	off	IORING_SETUP_SINGLE_ISSUER	none
`--uring-defer-taskrun`	off	IORING_SETUP_DEFER_TASKRUN	single issuer
`--uring-coop-taskrun`	off	IORING_SETUP_COOP_TASKRUN	none
`--uring-register-ring-fd`	off	Registers the ring descriptor to cut io_uring_enter cost	none

Flag	Baseline	On	Needs
`--uring-poll-first`	off	Polls before attempting a recv or send	none
`--uring-send-zc`	Copying send	Zero-copy send. Each send produces two completions, and the buffer may not be reused until the second (the notification) arrives, so the module keeps at least two write buffers per connection and hands the app a free one through io_get_buf. Useful for large sends only	none
`--uring-sqpoll`, `--uring-sqpoll-cpu N`, `--uring-sqpoll-idle MS`	off	A kernel thread polls the ring, so submit calls disappear but a core is burned; report its CPU separately	none
`--uring-async-file-read`	pread	io_read_file uses IORING_OP_READ (file and web)	none
`--uring-splice`	off	io_send_file moves file data to the socket through a pipe with splice (file and web)	none

Rough kernel needs (from memory; verify): provided buffer rings and multishot accept 5.19 or newer; multishot recv, single issuer and send-zc 6.0 or newer; defer-taskrun 6.1 or newer. **[SETUP]** Check `uname -r` on both machines and drop any flag the kernel lacks.

6.5 File and web flags

Flag	Baseline	On
`--sendfile`	io_read_file (pread) into the buffer, then send	io_send_file uses sendfile() in the kernel (select, poll, epoll builds); the header is sent first with MSG_MORE so it travels with the file data
`--file-read=mmap`	pread	Maps the file instead of copying it into the buffer

Apache is not configured from the command line the same way, so its owner exposes these same flag names through one environment variable read at startup (for example `APACHE_IO_FLAGS`). **[CONFIRM]**

6.6 Which experiment tests which flag

Every flag has a home. Tier 1 flags are always tested. Tier 2 flags are tested if time allows. Tier 3 flags are implemented last, and only if time remains; otherwise remove them from the build rather than leave untested code.

Tier	Flags	Tested in
1	`--epoll-et`	Echo E1, E2; file E5a; Apache E6a, E6b
1	`--accept-batch`, `--uring-multishot-accept`	Connection-heavy tests: E1 at 5,000 and 10,000 connections, E5b, E6c. for readiness modules it is mostly useful as `--accept-batch 1` to show the cost of not draining, and for io_uring as the accept depth.
1	`--uring-provided-buffers`	Echo E1, E2 (memory and cache), file E5a
1	`--uring-multishot-recv`, `--uring-fixed-files`	Echo E1 (syscalls and CPU per message)

Tier	Flags	Tested in
1	`--uring-single-issuer` with `--uring-defer-taskrun`	Echo E1 (CPU per message)
1	`--sendfile`, `--uring-splice`, `--uring-async-file-read`	File E5a, Apache E6d
2	`--epoll-maxevents`, `--uring-submit-batch`, `--uring-fixed-buffers`	Echo E1 high end
2	`--uring-send-zc`	File E5a, files of 1 MB and up
2	`--rcvbuf`, `--sndbuf`, `--chunk-size`	E7
2	`--uring-sqpoll` and its options	Echo E1 (CPU reported separately)
3	`--epoll-oneshot`, `--uring-coop-taskrun`, `--uring-register-ring-fd`, `--uring-poll-first`, `--busy-poll`, `--backlog`, `--file-read=mmap`, `--uring-entries`, `--uring-cq-entries`	Only if time remains

6.7 How flags are used in experiments

Turn flags on one at a time, always against a baseline run from the same session. The io_uring ladder is cumulative: fixed files, then fixed files plus provided buffers, then plus multishot, then plus single issuer and defer-taskrun. Compare each step with the one below it, and also try each flag alone wherever it works. Re-run the correctness gate every time a flag is added.

7. Instructions: echo server owner

You build apps/echo.c, its tests and its results. The echo server returns every fixed-size frame it receives, unchanged. It is the simplest server, so it isolates the notification mechanism better than the other two.

7.1 What to build

1. Write apps/echo.c against the plug-in contract (section 4). It contains only echo logic and no mechanism-specific code.
2. `on_accept` calls `io_read(conn, buf-size)`.
3. `on_read_done(conn, buf, n)`: if n is 0 the client closed, so close the connection and discard any half-received frame without echoing it. Otherwise treat the carry slot plus the new bytes as one stream, find every complete frame of `--msg-size` bytes, and send all of them back with one io_write (assemble them in a buffer from io_get_buf when a carry is involved). Copy any trailing partial frame into the carry slot. When `on_write_done` fires, call io_read again.
4. Reading a whole buffer at a time and echoing every complete frame is how a normal stream server works. TCP can therefore bundle several frames into one read, so count frames per read in the stats table, and compare syscalls per message only between runs at the same load (7.3).
5. Count frames echoed, frames per read, bytes in and out, reads, writes, partial writes, `errors` and `peer_resets` (section 5.1).
6. Flags that apply: `--msg-size`, `--nodelay`, `--buf-size`, `--accept-batch`, plus every module flag in section 6.

7.2 Tests to pass first

The full gate is in section 13. For echo, every test applies: split frame, coalesced frames, one byte at a time, a full-pipe stall, close mid-frame, reset mid-frame, 10,000 connect-echo-close cycles with a descriptor-leak check, 1,000 concurrent connections, random chunking, half-close, and a slow reader.

7.3 Experiments to run

Load generator: tcpkali (section 11). **Offered load policy.** Every experiment except E1s uses a fixed total message rate R, split equally over the active connections as a per-connection rate (tcpkali sets this per connection). R is chosen in the pilot at about half of the slowest mechanism's capacity **[SETUP]**, and the per-connection rate must be low enough that at most one message is in flight per connection (rate times round-trip time well below 1). This keeps latency meaningful, stops TCP bundling messages in the socket buffers, and means a connection-count change never changes the total load. Unthrottled sending is used only in E1s, where only throughput and CPU per message are reported, never latency, because a flooded connection measures socket-buffer queueing and not the mechanism.

Exp	Question	Fixed	Varied
E1 Connection scaling	How does cost grow with total connections at the same load?	512 B, total rate R, all connections active	Total connections 10, 100, 500, 1000, 5000, 10000 (select stops at 1000)
E1s Saturation	What is the maximum throughput?	512 B, unthrottled	Same connection counts; report throughput and CPU per message only
E2 Active ratio	Do idle connections hurt?	Total rate R held constant, so each active connection's rate is R divided by the number of active ones	Active share 1%, 10%, 50%, 100% of 1,000 (all four) and 10,000 (poll, epoll, io_uring) total connections
E3 Load versus latency	What latency at a given load?	1,000 connections	Total rate from 10% up to saturation, on a common grid up to the weakest mechanism's capacity, then continued per mechanism **[SETUP]**
E4 Message size	Syscall-bound or copy-bound?	100 connections, total rate adjusted to stay under the wire limit	64 B, 512 B, 4 KB, 16 KB
E7 Socket buffer sensitivity	Do kernel buffer sizes change the picture?	1,000 connections, unthrottled	`--rcvbuf` and `--sndbuf` at 64 KB, 256 KB, 512 KB against the default

For E2 and E6b, open the idle and active client instances at the same time with connect rates proportional to their counts, so that active and idle connections interleave in accept order instead of the active ones sitting together in one block of the connection table.

7.4 Extensions to test for echo

Each against a baseline run from the same session, after re-running the tests. The tiers are in section 6.6.

Flag	Where it should show
`--epoll-et`, `--epoll-maxevents`	Many active connections; E1 high end
`--accept-batch`, `--uring-multishot-accept`	Time to accept 5,000 and 10,000 connections; E1 ramp
`--uring-fixed-files`	CPU per message at the same load
`--uring-provided-buffers`	Many idle connections; memory and cache misses (E2). The app copies split frames into the carry slot, so frames spread over several buffers are handled
`--uring-multishot-recv`	Syscalls per message. A completion can hold several frames or part of one
`--uring-single-issuer`, `--uring-defer-taskrun`	CPU per message
`--uring-submit-batch 1` versus baseline	Value of batching submissions
`--uring-fixed-buffers`, `--uring-sqpoll`	CPU per message (SQPOLL CPU reported separately)

Three connection counts (low, medium, high) are enough per flag.

7.5 Common mistakes

- Dropping or overwriting bytes that arrived behind the frame being handled; keep them in the carry slot.
- Echoing a frame before all of its bytes have arrived.
- Closing a connection while an io_uring request on it is still in flight; keep a generation counter in the connection table.
- Reusing a descriptor number and mixing up old and new connections.

- Client and server using different message sizes.
- Counting a reset at client shutdown as a server error.

Done when: all tests pass on all four builds, E1 to E4, E7 data is accepted under section 12, and extension results are written up.

8. Instructions: file-transfer server owner

You build apps/filesrv.c, its tests and its results. The server streams a requested file to the client. It stresses large writes and backpressure, which echo barely touches.

Protocol decision: the request and response use a minimal HTTP/1.1 subset so that existing load generators (wrk, wrk2, oha, curl) can drive it without custom code. **[CONFIRM]** The alternative is a custom request line driven by tcpkali, which counts bytes received but cannot check content.

8.1 What to build

1. **Request:** 1. **HTTP protocol:** Support a minimal HTTP/1.1 subset. Requests use `GET /<name> HTTP/1.1` followed by optional headers and a terminating blank line. The maximum request size is 4 KB; reject oversized or malformed requests with `400 Bad Request`. Only GET is supported; other methods receive `405 Method Not Allowed`. Successful responses contain `HTTP/1.1 200 OK`, `Content-Length`, and the appropriate `Connection` header, followed by a blank line and the file bytes. Missing files receive `404 Not Found`. Support `Connection: keep-alive` and `Connection: close`, closing the connection after the response when requested. Resolve requested filenames against the fixed startup file table; reject paths containing `..` and any filename not in the table.
2. **Open the files once, at startup.** The document root holds a fixed set of at most 16 files. Open each one, keep the descriptors and the sizes in a table, and answer every request with a table lookup and pread at the connection's own offset. Never call open, fstat or close per request. Reasons: the benchmark should measure network I/O and not path lookup and file-table work, and holding one descriptor per transfer would make select fail near 500 concurrent downloads, because each transfer would use a socket and a file descriptor. With the table, 1,000 connections plus a handful of fixed descriptors stays under 1,024.
3. **For each request:** 3. ***File transfer sequence:** For each request, obtain an output buffer using `io_get_buf()` and write the HTTP response header at its beginning. Ensure the buffer has enough capacity for the header plus one file chunk. Call `io_read_file()` to read up to `--chunk-size` bytes (default 64 KB) directly into the buffer region immediately after the header. Wait for `on_file_read_done()` before calling `io_write()`. Then send the header and first file chunk together in a single `io_write()`. The buffer must remain valid and unchanged until `on_write_done()`. After each `on_write_done()`, advance the file offset and repeat the file-read and write sequence until exactly `Content-Length` bytes have been sent. Then process the next request on the same connection.
4. **Pipelined requests.** The read that delivers one request may also deliver the next ones. Keep the bytes behind the request being handled in the carry slot (section 4) and handle them before asking for more data. Never reuse the read buffer for a response while requests sit in it.

5. Files come from the OS page cache.5. ***File caching and reads:*** Warm the OS page cache before every benchmark run by reading each test file once, and verify page residency using `vmtouch` or `fincore`. This reduces storage I/O as a source of variation. Regular files do not provide meaningful asynchronous readiness through `select`, `poll`, or `epoll`; therefore, all four baseline implementations use the same synchronous `pread()` for file reads. Asynchronous file reads through `io_uring` are evaluated separately using `--uring-async-file-read` (Section 15).
6. Backpressure is the main correctness risk: a slow reader on a 100 MB file must pause only its own connection, and the server's memory must stay bounded. Select, poll and epoll keep an output slot only for connections whose send is incomplete.
7. Generate the test files once with a fixed random seed: 4 KB, 64 KB, 1 MB and 100 MB, and keep a SHA-256 manifest. Use the same files on every run.
8. Flags that apply: `--chunk-size`, `--buf-size`, `--nodelay`, `--accept-batch`, `--sendfile`, `--file-read=mmap`, `--uring-async-file-read`, `--uring-splice`, `--uring-send-zc`, plus the common and module flags in section 6.

8.2 Tests to pass first

The gate in Section 13, plus:

* SHA-256 matches for every test file size (download using curl and compare against the manifest).

* Missing file returns 404.

* Path traversal attempts are rejected.

* Zero-length file is transferred correctly.

* Client aborts mid-transfer; no resources leak.

* Sequential requests over a keep-alive connection work correctly.

* Slow reader downloading the 100 MB file does not block other clients.

* Many concurrent downloads complete correctly.

8.3 Experiments to run

Load generator: wrk (closed loop) and oha or wrk2 for fixed rate; curl with `--limit-rate` for slow readers (section 11). Run in the default delay setting.

Exp	Question	Varied
E5a Size and concurrency	How do large writes and backpressure behave?	File size 4 KB, 64 KB, 1 MB, 100 MB; concurrent downloads 1, 10, 100, 1000 (select stops at 1000)
E5b Connection reuse	Does accept cost matter?	One connection per file versus keep-alive reuse, small files. Check TIME_WAIT behavior in the pilot (section 14)
E7 Buffer sensitivity	Do chunk and socket buffer sizes matter?	`--chunk-size` 16 KB, 64 KB, 256 KB; `--rcvbuf` and `--sndbuf` 64 KB, 256 KB, 512 KB against the default
E8 Slow readers	Does a mechanism cope when some clients read slowly?	100 fast downloaders (wrk) plus 0, 10 and 100 slow readers (curl, 100 KB/s) on the 100 MB file; report the fast clients' throughput and p99, and the server's peak memory

For files of 64 KB and up at modest concurrency the wire may fill first **[SETUP]**. In that case the headline numbers are server CPU per MB, syscalls per MB and completion-time percentiles, not MB/s.

E5a: "Select's maximum concurrency depends on FD_SETSIZE and actual descriptor values, not a fixed limit of 1000."

E5b: "Connection reuse means sequential requests over keep-alive, not HTTP pipelining."

E8: "A 100 MB download at 100 KB/s takes approximately 16 minutes 40 seconds. Is this experiment duration practical?"

E9: "Since header and first chunk are combined into one write, Nagle's effect may be smaller. Still worth measuring."

8.4 Extensions to test for file transfer

--sendfile: "How will the application communicate MSG_MORE through the current io_write() interface?"

--uring-send-zc: "Ensure buffers remain valid until the zero-copy notification, not merely the initial send completion."

Missing flag: Add --uring-provided-buffers to the 8.1 flag inventory, since it appears here.

Flag	Where it should show
`--sendfile` (select, poll, epoll) and `--uring-splice` (io_uring)	CPU per MB on large files. With `--sendfile` send the header first with MSG_MORE, because sendfile cannot carry in-memory bytes and the header would otherwise leave as a separate small packet
`--uring-async-file-read`	CPU per MB and syscalls per MB on large files
`--uring-send-zc`	Large files; the module keeps two or more write buffers per connection, so memory per connection rises
`--uring-provided-buffers`	Memory at high concurrency
`--chunk-size`, `--rcvbuf`, `--sndbuf`	E7
`--epoll-et`, `--accept-batch`, io_uring ladder from section 6	As in section 7.4

8.5 Common mistakes

8.5 Common mistakes

* Content-Length not matching the number of bytes actually sent.

* Sending the header and body as separate small writes in the baseline.

* Using sendfile without MSG_MORE on the header when more file data follows.

* Calling open, fstat or close per request.

* Some runs using a cold page cache or a different file set.

* Treating a short write as complete.

* Reading the next file chunk before the previous write completes.

* Reusing a buffer while an asynchronous operation still uses it.

* Allowing unbounded buffering for slow clients.

Done when: All tests pass on all four builds, E5, E7, E8 and E9 results are accepted under Section 12, and extension results are documented.

9. Instructions: Apache / web server owner

Apache httpd is existing, multi-threaded code, so you do not write an app file. You patch Apache so its event-waiting layer can use each mechanism, then benchmark the variants against each other. Because Apache is not single-threaded like our other servers, compare Apache variants with each other, never directly with echo or file-transfer results. We do not change select, poll, epoll or io_uring themselves: the work is in Apache's and APR's wrappers around them.

9.1 What to know first

These points come from the Apache and APR documentation and source as we understand them. Verify each in the exact version you build, with the source and with strace. **[CONFIRM]** httpd and APR versions.

- Apache does not call epoll directly. It goes through APR's pollset layer, which has select, poll and epoll methods (and kqueue and port on other systems). APR has a flag that stops it silently falling back to a default method when yours cannot be used. Always set it, otherwise an apparent select build may really run on epoll.
- Only the **event MPM** multiplexes connections this way. A dedicated listener thread polls the listening sockets and connections waiting between requests (keep-alive) or waiting to finish an asynchronous write. Worker threads handle the request itself. The prefork and worker MPMs do not use a shared pollset for connections, so they cannot show a mechanism difference. The event MPM keeps a short list of acceptable pollset methods in event.c, starting with kqueue, and its documentation says it needs epoll or kqueue.
- **select and poll are likely to need real work.** As we understand it, the event MPM creates its pollset as thread-safe and no-copy, because worker threads add and remove sockets while the listener thread waits, and APR's select and poll backends are understood to refuse those flags. Check srclib/apr/poll/unix/select.c and poll.c. If that is right, editing the method list is not enough: you would have to add locking, pointer tracking and wake-up support to those two backends, or the server will corrupt memory. This is plumbing around the mechanism, not a change to select or poll. **[CONFIRM]** whether the team takes this on; if not, Apache is compared only with epoll and io_uring and the report says why.
- **Apache uses descriptors itself.** Before any client connects Apache holds a few dozen descriptors (listeners, logs, pipes, scoreboard), and worker threads may open files for the responses they serve. A select variant can therefore hit the 1,024 limit well before 1,000 clients. Find the highest working connection count in the pilot and treat that as select's result.

- **Worker threads may call poll() on their own.** As far as we know, APR can wait on a single socket by calling poll() directly, outside the pollset. If so, strace will show poll() in every variant, including the epoll and io_uring builds. Count those calls separately; they are the same in every variant and are part of how Apache works.
- The APR method list we checked has no io_uring. You add a backend. A pollset is a readiness API, so io_uring can only be used here as a notifier (poll-add), not in the completion style of our own servers, and the report must say so. **[CONFIRM]** whether the team attempts completion-style io_uring inside Apache (the default is no; it would mean rewriting how Apache's workers read and write sockets).
- **The io_uring submission queue is not thread-safe.** Several worker threads call pollset add and remove while the listener waits. Either put a mutex around every submission, which rules out `--uring-single-issuer` and `--uring-defer-taskrun`, or send all adds and removes through a queue that only the listener thread turns into ring operations, which keeps those two flags possible. **[CONFIRM]** which approach.

9.2 What to do

1. Build httpd from source with the event MPM and the bundled APR. Record versions and configure flags. Use identical compiler flags for every variant.
2. Make the mechanism selectable at startup through an environment variable (for example `APACHE_IO=select|poll|epoll|uring`), and pass the flags from section 6 that make sense through `APACHE_IO_FLAGS`. **[CONFIRM]** names.
3. In event.c, force the chosen method and set the no-fallback flag. Confirm with strace that each variant uses the expected call (select, poll, epoll_wait or io_uring_enter) and record any poll() calls that come from elsewhere.
4. For io_uring, add a pollset backend (a new file next to the epoll one in APR) built on IORING_OP_POLL_ADD, using the thread-safety approach decided above. The multishot poll option is the relevant extension here.
5. Configure Apache the same way in every variant: event MPM, one process, a fixed ThreadsPerChild **[SETUP]**, ServerLimit, ThreadLimit and MaxRequestWorkers raised high enough for the connection counts being tested, a long KeepAliveTimeout so idle connections stay open, access logging off, unneeded modules off. Fix EnableSendfile to one value for every variant (Off as the baseline, On as an extension). Record the whole configuration with every run.
6. Serve static files from the same document root: `/small` (1 KB), `/medium` (64 KB), `/large` (1 MB). Warm the page cache before every run.
7. Pin the Apache process to the same set of cores in every variant **[SETUP]**. Record the number of cores.
8. Delay versus no delay: see section 5.2 for what Apache sets by default.

9.3 Tests to pass first

Curl every path and compare body hashes; request split into pieces and sent one byte at a time; pipelined requests; malformed and oversized headers; keep-alive reuse; `Connection: close`; a wrk

run reports zero non-2xx responses; a slow reader on `/large`; and the descriptor count returns to its starting value after many connections open and close.

9.4 Experiments to run

Load generators: wrk, wrk2 or oha, curl with `--limit-rate` for slow readers, and tcpkali to hold idle connections (section 11). Run in the default delay setting.

Exp	Question	Varied
E6a Connection scaling	How does cost grow with connections?	10, 100, 500, 1000, 5000, 10000 connections on /small with keep-alive; for select use 10, 100, 500 and the highest count that works
E6b Active ratio	Do idle keep-alive connections hurt?	Active share 1%, 10%, 50%, 100% of 1,000 and 10,000 connections, total request rate held constant
E6c Connection per request	How does accept cost behave?	Keep-alive on versus off. Check TIME_WAIT behavior in the pilot (section 14)
E6d Response size	Syscall-bound or copy-bound?	/small, /medium, /large
E6e Load versus latency	What latency at a given load?	Fixed rate steps up to saturation **[SETUP]**
E8 Slow readers	Does the variant cope when some clients read slowly?	100 fast clients plus 0, 10 and 100 slow readers on /large

The pollset is used on every request that passes through the keep-alive or asynchronous-write states, not only for idle connections, so differences may show under busy load too; check the number of pollset calls per request with strace or perf. Small differences are a finding, not a failure.

Reference check: run nginx with one worker pinned to one core on /small. It uses epoll, so its numbers should be the same order of magnitude as Apache with epoll. If not, investigate before trusting any Apache result.

Done when: all variants pass the tests, E6 and E8 data is accepted under section 12, and the report states exactly what each variant does and what it does not.

10. Instructions: I/O module owners

You write the four files in io/. Each one implements the same contract (section 4), and the apps never see what is inside. All three server owners depend on your work, so finish the baselines first and run them with the echo app before adding any flag. Write each mechanism the way it is normally used. Do not add tricks that make it faster or slower than a normal server using it would be.

10.1 Rules for all four modules

- Implement the contract exactly. Callbacks are called only from inside io_run, never from inside an io_ call. If a request finishes at once, put its completion on an internal list that io_run delivers before it waits again, using a zero timeout while the list is not empty. This avoids unbounded recursion.
- Keep per-connection request state: pending read (maximum), pending write (buffer, length, offset), and a small connection record. Partial writes are finished inside the module.
- Buffers: select, poll and epoll read into one shared scratch buffer of `--buf-size`, and when a send is incomplete they copy the unsent bytes into an output slot from the pool, released when the write finishes. io_uring uses one buffer per connection (or the provided pool with its flag). Do not give the readiness modules per-connection buffers.
- All sockets are non-blocking. Peer close is reported as on_read_done with n equal to 0; other failures go to on_close.
- Keep these counters in the shared stats table: loop iterations, wait calls (select, poll, epoll_wait or io_uring_enter), reads, writes, EAGAIN, partial reads, partial writes, accepts, closes, `errors`, `peer_resets`. They let you cross-check perf and strace syscall counts later. Counters are plain integers; the orchestrator subtracts a snapshot taken before the measured run from one taken after (section 12).
- Accepting: readiness modules drain the accept queue until EAGAIN by default; io_uring keeps one ACCEPT outstanding and re-arms it. With `--accept-batch N`, readiness modules accept at most N per iteration and io_uring keeps N ACCEPT requests outstanding.
- Do no hidden optimization. If something is faster than the normal way, it is a flag, not part of the baseline.
- Print the active flag set at startup and refuse invalid combinations with a clear error.
- A module owner from another module reviews each module before it is accepted.

10.2 select (io_select.c)

- Rebuild the read and write sets from the connection list on every loop iteration: a connection with a pending read goes in the read set, one with a pending write in the write set, and the listener in the read set.
- Track the highest descriptor. Refuse any descriptor at or above 1,024. Remember that standard streams, the listener and any fixed files use descriptors too, so the practical limit is a little under 1,024.
- After select returns, scan every registered connection and test its bit. This full scan is the cost the benchmark is meant to expose, so do not replace it with an active list.

10.3 poll (io_poll.c)

- Keep a contiguous pollfd array with the listener first. On close, swap the last entry into the gap and update the index map.
- Use POLLIN for a pending read and POLLOUT for a pending write. Handle POLLERR, POLLHUP and POLLNVAL.
- After poll returns, scan the whole array, as the normal version does.

10.4 epoll (io_epoll.c)

- epoll_create1, then add each accepted socket once with EPOLLIN, level-triggered. Do not modify the registration on every message.
- When the app calls io_write, try send() at once. If it sends everything, the write is complete and no epoll_ctl call is made. Only when send() comes back incomplete or with EAGAIN, copy the rest into an output slot and make one modify call to wait for EPOLLOUT instead of EPOLLIN, so a slow client is not read from while its output is stuck. When the output is finished, make one modify call back to EPOLLIN. In a normal run with healthy clients this means zero epoll_ctl calls per message.
- epoll_wait with maxevents 256 and no timeout (zero timeout while completions are queued). Handle EPOLLRDHUP, EPOLLERR and EPOLLHUP. The ready connections of each wait are handled one by one.
- Flags: `--epoll-et` reads and writes until EAGAIN and keeps a per-connection flag for connections whose read filled the buffer without reaching EAGAIN, revisiting them each iteration, because an edge-triggered epoll will not report data it has already reported; `--epoll-maxevents` changes the batch; `--epoll-oneshot` re-arms after every event. //should be absolutely implemented
- When `io_write` is called in `io_select`, `io_poll`, or `io_epoll`, the module attempts one immediate non-blocking `send()`. If all `len` bytes are sent, it marks the write complete and fires `on_write_done` via a deferred ready list at the end of the current event step (avoiding recursive callbacks). Only if `send()` returns `EAGAIN` or a partial write does the module register for writability (`EPOLLOUT` / `POLLOUT` / write `fd_set`)
- Under `--epoll-et`, if `recv()` returns `n == max` (buffer full, did not hit `EAGAIN`), the socket stays in a module-level `et_ready_list` so the next `io_read()` processes the remaining kernel bytes immediately without waiting for a new `epoll_wait` edge

10.5 io_uring (io_uring.c)

- Use liburing. Size the submission ring per run to at least the number of connections (next power of two) and leave the completion ring at liburing's default of twice that. A 4,096-entry ring is too small for 10,000 connections. After each run check the ring's overflow flag and counter; any overflow rejects the run (section 12.4)

  //i think we should do the following for the baseline.:

3. What I would choose for your benchmark

Recommended baseline

Decision	Choice	Reason
SQ	16,384	Room for large submission batches
CQ	32,768	More headroom for completion bursts
CQ draining	Regular batched draining	Prevent accumulation
In-flight operations	Track explicitly	Understand actual workload
Overflow	Check `IORING_FEAT_NODROP`, handle overflow and `-EBUSY`	Correctness and robustness
Dynamic resizing	Not initially	Unnecessary complexity for fixed workload

Also keep this:
For a fair benchmark, record:

- **Maximum outstanding SQ submissions.**
- **Maximum CQ occupancy.**
- **Number of CQEs processed per event-loop iteration.**
- **Whether overflow occurred.**
- **Whether submission ever returned `-EBUSY`.**

- Encode user_data as connection id, generation counter and operation type.
- Keep ACCEPT requests outstanding (one by default) and re-arm on completion. When the app calls io_read, queue a RECV into the connection's buffer; when it calls io_write, queue a SEND. A partial send queues another SEND for the rest.
- Call submit-and-wait once per loop iteration, so everything queued while handling completions goes in one batch.
- If the submission queue is full, submit and retry.
- Never free a connection or its buffer while a request on it is in flight: use the generation counter, and on close either cancel the request or wait for its completion.
- Each flag is a separate code path chosen at startup. Provided buffers need buffer recycling, and a buffer returns to the ring only after its data has been used or sent. Multishot completions carry a more-to-come marker and must be re-armed when it clears, including when buffers run out. Send-zc needs the two-completion handling and at least two write buffers per connection (section 6.4). Defer-taskrun needs the loop to fetch completions explicitly. Sqpoll needs the submission-queue wakeup check. Fixed buffers need one contiguous arena and enough locked memory. io_read_file and io_send_file use async file read and splice when their flags are on.

10.6 First tests

Run the echo app on each module with 1, 100 and 1,000 connections, and check that the counters make sense: for example, reads and writes per frame match what the module should do, and the epoll module makes no epoll_ctl calls per message on healthy connections. Then run the correctness gate (section 13) before anyone benchmarks.

Done when: four baseline modules pass the gate with the echo app, and every flag in section 6.6 that is in Tier 1 or Tier 2 is implemented, off by default, and covered by a gate re-run.

11. Load generators and what to measure

We use existing load generators only, chosen per server. The tool must be able to push more load than the server can handle, or we would be measuring the tool.

11.1 Which tool for which server

Server	Fixed-rate load (most experiments)	Maximum throughput (E1s, E5a)	Idle connections, slow readers, integrity, cross-checks
Echo	tcpkali with a per-connection message rate, so each connection	tcpkali unthrottled,	A second tcpkali instance with no messages holds idle connections; `netperf`

	has at most one message in flight; latency through a latency marker	throughput and CPU only	`TCP_RR` for a lockstep cross-check at a few connections
File transfer	wrk2 or oha with a fixed request rate	wrk (closed loop, multi-threaded)	curl plus sha256sum for integrity; curl with `--limit-rate` for slow readers; iperf3 only to measure the wire ceiling
Apache	wrk2 (fixed rate, built to avoid coordinated omission) or oha, which has a latency-correction option and an option to disable keep-alive for connection-per-request tests	wrk	tcpkali holds idle connections; curl with `--limit-rate` for slow readers; `ab` only as a sanity check, since it is single-threaded

Notes:

- tcpkali measures latency by watching for a marker string in the returned data, which fits an echo server. It sends continuously unless told a rate, so never report latency from an unthrottled tcpkali run: with the socket buffers full, the number would be queueing delay in the kernel, not the cost of select, poll, epoll or io_uring. Check the man page for exact behavior.
- Lockstep load at many connections has no standard tool. A low per-connection rate does the same job: when rate times round-trip time is well below 1, each connection has at most one message in flight.
- Delay versus no delay applies to the client too (section 5.2): tcpkali has a Nagle option, while wrk and wrk2 appear to have none. **[CONFIRM]** each tool's behavior and record it in the run configuration.
- Existing tools have no warm-up of their own. Run one warm-up invocation (results discarded) and then the measured invocation against the same running server (section 12).
- When a load generator finishes, it closes sockets that may still hold unread data, and the kernel then sends resets. The server counts these as `peer_resets`, which is expected and not a failure (section 5.1).
- Client machine health: client CPU under 80% on every core, zero client-side errors, zero non-2xx responses. If a core is above that, add client threads or processes. **[SETUP]** client core count.
- Connection-per-request tests (E5b, E6c) can run out of client ports or pile up TIME_WAIT sockets; see section 14.

11.2 What to measure

Metric	How	When
Throughput (messages per second, MB per second)	Load generator output	Measured invocation
Latency p50, p90, p99, p99.9 and max	Load generator histogram, fixed-rate runs only	Measured invocation
Server process CPU: user and system	cgroup v2 `cpu.stat` (usage_usec, user_usec, system_usec) of a dedicated cgroup that holds the server, read before and after. In recent kernels io_uring worker and sqpoll threads belong to the server process, so their time is included; confirm this in the pilot against per-core /proc/stat	Measured invocation
Network receive and interrupt CPU	/proc/stat per CPU, softirq and irq columns, for the NIC interrupt core (and the server core as a cross-check), before and after. Do not sum all cores: other cores add unrelated noise	Measured invocation
CPU microseconds per message	(cgroup CPU delta + softirq and irq time on the interrupt core) divided by completed messages or MB	Derived
Peak memory	VmHWM from /proc/PID/status, socket memory from /proc/net/sockstat	After the run
Syscalls per message	`perf trace -s -p PID` (summary by syscall), or `perf stat -e 'syscalls:sys_enter_*'`, or a bpftrace script counting by syscall id, divided by completed messages. Note that `perf stat -e raw_syscalls:sys_enter` gives one combined total, not a breakdown. Cross-check with	Separate diagnostic run

	the module counters. Use `strace -c` only for the syscall mix, never for timing	
Frames per read	Server counter	Every echo run
IPC, cache and LLC misses, context switches, migrations, page faults	`perf stat`	Separate diagnostic run
Where time goes	`perf record -g` flame graphs at selected configurations	Separate diagnostic run
Per-call cost of select, poll and epoll_wait versus number of descriptors	`perf trace` or bpftrace with few active connections; the direct evidence for linear versus constant scaling	Separate diagnostic run
io_uring ring overflow	The ring's overflow flag and counter, checked after the run	Every io_uring run
CPU frequency and temperature	Read before and after each run	Never during

No sampling during a measured run: counters are read before and after only, and the difference is used (P6).

12. How to run an experiment

Every experiment in sections 7 to 9 follows the same protocol: 7 repeated runs per configuration, 30 seconds measured after a 10 second warm-up, and the median of the 7 as the reported number. The values below are defaults and are frozen only after the pilot (12.5). **[SETUP]**

12.1 Words used

- **Configuration:** server, mechanism build, flags, and one workload point. Example: echo, epoll, no extra flags, 1,000 connections, 512 B, delay.
- **Run:** one execution of a configuration.
- **Cell:** a configuration with all its repeats, reported as one median.

12.2 Defaults

Setting	Default	Notes
Warm-up	10 s	Results discarded
Measured window	30 s	Counters read immediately before and after, and the difference used
Cool-down between runs	10 s	Lets the CPU settle
Repeats per configuration (R)	7, minimum 5	The median of 7 values is the 4th smallest
Server process	Fresh for every run	Pinned, in its own cgroup, flag set printed at start
Delay setting	Default (no delay) everywhere	Recorded with every run

12.3 One run, step by step

1. Apply the machine tuning script on both machines and record the environment (section 14). **[SETUP]**
2. Start a fresh server process with the configuration's flags, inside its cgroup.
3. Warm-up invocation: run the load generator for 10 s and discard its output. Then let it finish and wait for the connections to close.
4. Start the measured invocation. Wait until the server's accept counter shows that every connection has been accepted (section 5.1).
5. Take snapshots on the server machine: the server's counters (SIGUSR1), the cgroup cpu.stat, /proc/stat for the interrupt core and server core, CPU frequency and temperature.
6. Let the measured invocation run for 30 s and save its output. For tools that connect at start, keep connection set-up short compared with 30 s, using the tool's ramp option for high connection counts.
7. Take the same snapshots again. Every server counter and CPU figure used in the results is the difference between the second snapshot and the first, so the warm-up never leaks into the measured numbers.
8. Stop the server and wait out the cool-down.

12.4 Order and statistics

- **Order:** group runs into blocks. One block holds every mechanism for one workload point, once, in random order. Repeat blocks until each configuration has R runs. This spreads heat and background noise over all mechanisms instead of favoring whoever runs first.
- **Per run:** compute throughput, latency percentiles, CPU per message, memory and syscalls per message.
- **Across the 7 runs:** report the median as the headline value, the interquartile range as the spread, and the minimum and maximum. For latency percentiles, take the percentile inside each run and then the median of those 7 values. Never average percentiles and never mix requests from different runs.
- **Is a difference real?** Compare the 7 per-run values of the two configurations with a two-sided Mann-Whitney U test at 0.05 (with 7 runs each the smallest possible p-value is about 0.0006, so tight, non-overlapping results show clearly). Always report the percentage difference between the medians next to each configuration's spread. There is no fixed percentage cutoff: a small but consistent difference is reported as small and consistent, and whether it matters in practice is argued in the discussion.
- **Rejecting a run:** re-run it if any of these happen: the server's `errors` counter is above zero; the client reports errors (non-2xx responses, connect failures, content mismatches, timeouts); `peer_resets` is higher than the number of connections opened in the run; the client health check fails (section 11); the server crashes; the io_uring completion ring overflowed; the kernel logs warnings; or CPU frequency dropped more than 5% during the run. Short reads and resets at client shutdown are normal and never a reason to reject.
- **Noisy cells:** if throughput varies by more than 5% (standard deviation divided by mean) across the 7 runs, investigate, add repeats up to 15, and if it is still noisy say so in the report.

12.5 Pilot first

Before any real experiment, do short pilot runs (10 s). They are for debugging and are never reported. The pilot decides: the warm-up and measured lengths (check that the per-second throughput is flat by the end of warm-up), the number of repeats (run one configuration 10 times and look at the spread), client threads, which machine is the client, the total offered rate R and the rate steps for the load-versus-latency experiments, the buffer sizes, whether the wire or a CPU saturates first, and whether connection churn runs out of ports.

With one accept per iteration, io_uring may need noticeably longer to ramp to 5,000 or 10,000 connections, so set a generous ramp timeout.

[SETUP]

12.6 Baseline versus extensions

Every flag run is paired with a baseline run from the same session, never with an old result. Test each flag at three connection counts (low, medium, high) with R repeats, and report its effect as a percentage change in the median with the spread. Re-run the correctness gate before benchmarking a new flag.

12.7 Time budget

Total time is roughly: configurations x R x 50 s, times 2 if an experiment is run both with and without delay. Worked example for echo E1: 6 connection counts for poll, epoll and io_uring plus 4 for select is 22 configurations, times 7 runs, times 50 s is about 7,700 s, or 2.1 hours, plus start-up time. Echo E9 (3 workload points, 2 delay settings, 4 mechanisms) is 24 configurations, about 8,400 s or 2.3 hours. If the team decides to run every experiment both with and without delay, double each experiment's time. Compute this for your own experiments before you start, and cut experiments in this order if time runs short: message-size sweep, Tier 3 and then Tier 2 flags (section 6.6), E7 buffer sensitivity.

12.8 What to keep for every run

One folder per run under results/, named `server-mechanism-flags-workload-runN`, containing: the configuration file, the git commit hash, the environment dump (uname, lscpu, ethtool, sysctl values, governor, offloads, interrupt placement), the raw client output, both counter snapshots, the cgroup and /proc/stat snapshots, and the delay setting used on server and client. One script (tools/run_experiment) performs the whole protocol and writes this folder, so no step depends on memory.

13. Correctness gate

No benchmark result counts until all four builds of a server pass this gate, and it is re-run whenever a flag is added or the code changes. A server that fails it may look fast only because it is wrong. The tests are written for echo; file and web owners adapt each one to their protocol and add the extras listed in sections 8 and 9.

ID	Test	Must hold
T1	One message sent in two halves 10 ms apart	No reply until the message is complete, then an exact reply
T2	Three messages in one send	Three exact replies, in order
T3	Bytes sent one at a time	Correct replies
T4	Client sends without reading until its own non-blocking send returns EAGAIN (every buffer on the path is full), waits 5 s, then reads everything back	No crash; server memory bounded and unchanged during the wait; every byte the server accepted comes back intact. The test cannot send a fixed 50 MB first, because once the buffers fill the client's own send would block forever, and that stall would come from the test, not the server
T5	Client closes mid-message	Clean close, no leak
T6	Client resets the connection mid-message	Clean close, no leak
T7	10,000 connect, use, close cycles	Descriptor count and memory return to baseline
T8	1,000 concurrent connections (and the 1,001st on select)	All correct; the extra connection is refused cleanly
T9	Random chunking of random data over many connections	Output equals input

T10	Client closes its sending side after sending	Server finishes replying, then closes
T11	The same recorded session against all four builds	Byte-identical output
T12	Sanitizer builds (ASan and UBSan) run T1 to T11	Clean
T13	File and web only: two and three requests pipelined in one send	Every response arrives, intact and in order
T14	Client sends continuously while reading slowly (for example 1 KB every 10 ms) for 30 s	No deadlock, no loss, bounded memory, output equals input

One command (`make test`) runs the whole gate for every build and saves a report.

14. Setup-dependent items

The hardware and network setup is not final. Everything marked **[SETUP]** in this plan depends on the items below. Fill the Value column in once, and the plan can be frozen.

Item	Value	Used for
Both machines: CPU model, cores and threads, RAM		Pinning, client threads, Apache thread count
Kernel version (`uname -r`) on both machines		Which io_uring flags exist (section 6.4)

gcc, liburing, Apache and APR versions	Reproducibility, Apache work (section 9)
`kernel.io_uring_disabled` value	Whether io_uring runs at all
Locked-memory limit (`ulimit -l`)	`--uring-fixed-buffers`
cgroup v2 available for the server process	CPU measurement (section 11.2)
`kernel.perf_event_paranoid` and perf permissions	Diagnostic runs
NIC type (PCIe, Thunderbolt, USB) and link speed	Wire ceiling, P5
Topology: two machines, loopback, or both; which machine is the client	Sections 5.4, 12.5
Core for the server, core for NIC interrupts, cores for the client, SMT handling	`--cpu`, fairness of CPU measurements
NIC interrupt placement: same core as the server or a different one; irqbalance stopped	Decide once, record, and keep identical for every run. Interrupts on the server core steal time from the event loop but keep packets in a warm cache; interrupts on another core cost cross-core cache traffic, and that core's softirq time is added to CPU per message

CPU governor, turbo setting, thermal behavior under load	Run stability (section 12)
sysctls: somaxconn, tcp_max_syn_backlog, ip_local_port_range, tcp_tw_reuse; ulimit -n	Connection counts up to 10,000
NIC offload and interrupt-coalescing settings (`ethtool -k`, `ethtool -c`)	Recorded as environment
Wire ceiling recomputed for the real link	Message sizes, which tests are wire-bound
Whether the wire fills first for file sizes of 64 KB and up	Headline metric for E5 and E6
Client threads needed to keep client CPU under 80%	Client health check
Total offered rate R, rate steps, warm-up, measured length, repeats	After the pilot (section 12.5)
Ring sizes used for io_uring runs at each connection count	Section 10.5
Apache ThreadsPerChild and cores; highest working connection count for the select variant	Section 9

Whether connection-per-request tests run out of client ports	E5b, E6c (see below)
Busy-poll support on the NIC	`--busy-poll`

Ports and TIME_WAIT in connection-per-request tests. The side that sends the first FIN keeps the connection in TIME_WAIT for 60 seconds. With `Connection: close` the server normally closes first, which frees the client's ports quickly. If the client closes first, its ports stay blocked, and with roughly 28,000 to 60,000 usable ports (depending on `ip_local_port_range`) only about 500 to 1,000 new connections per second are sustainable. tcp_tw_reuse only helps the side that holds TIME_WAIT, and only for outgoing connections. In the pilot, run E5b and E6c at the target rate for the full run length and check the client for connect failures. If ports run out, add several IP addresses to the client interface and spread connections over them, or lower the rate. **[SETUP]**

Commands to gather most of this on each machine:

uname -r

lscpu

ip -br link

ethtool IFACE

ethtool -k IFACE

ethtool -c IFACE

lsusb

sysctl kernel.io_uring_disabled kernel.perf_event_paranoid net.core.somaxconn net.ipv4.ip_local_port_range

ulimit -l

mount | grep cgroup2

systemctl status irqbalance

cat /proc/interrupts

cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor

15. Limits and open items

15.1 Differences that come with the mechanisms or the tools (to report, not remove)

- select cannot exceed 1,024 descriptors, and select and poll scan their whole descriptor set on every call. This is what we want to show.
- io_uring needs a buffer for every pending read until provided buffers are switched on, so its baseline uses more memory and cache at high connection counts.
- Readiness baselines drain the accept queue, while the io_uring baseline takes one connection per loop iteration, so io_uring is slower at absorbing bursts of new connections (E1 ramp, E5b, E6c, T7, T8). `--accept-batch` and multishot accept show how much of that batching recovers.
- Regular files cannot be polled, so file reads are synchronous in every baseline.
- In Apache, io_uring acts only as a readiness notifier, Apache is multi-threaded, APR may call poll() directly, and the select variant is limited by Apache's own descriptor use. Apache is compared only with other Apache variants.
- CPU is measured as the server's cgroup time plus interrupt-core softirq time, which still leaves out some work done on other cores; the method is the same for every run.
- Different load generators measure latency in slightly different ways and may set Nagle differently, so a result is compared only with results from the same tool and settings.
- Results depend on kernel version, hardware and laptop thermals. A 1 Gbps wire can hide differences for small messages and large transfers.
- Our servers are single-threaded; multi-core scaling is not studied.

15.2 Open decisions

- **[CONFIRM]** Owners for each role (section 1).
- **[CONFIRM]** File-transfer protocol: HTTP-style GET (planned) or a custom request.
- **[DONE]** the textbook drain loop for readiness modules.
- **[DONE]** Delay versus no delay: using no delay because load generators use no delay
- **[CONFIRM]** Apache and APR versions, the environment variable names for mechanism and flags, and what Apache sets for TCP_NODELAY by default.
- **[CONFIRM]** Whether the team extends APR's select and poll backends so the event MPM can use them.
- **[CONFIRM]** How the Apache io_uring backend handles several threads submitting (mutex, or a queue drained by the listener thread).
- **[CONFIRM]** Whether the team attempts completion-style io_uring inside Apache (default: no).
- **[CONFIRM]** Which Tier 2 and Tier 3 flags are built, and which flags are dropped because the kernel lacks them.

15.3 Sources

Checked by search on October 30, 2026; open the pages and verify before relying on any detail.

- tcpkali: connection counts, latency marker, echo use.
- oha: latency correction, fixed rate, keep-alive options.
- APR poll header: pollset methods and flags.
- Apache event MPM documentation: listener thread and keep-alive handling.

Appendix A. Brief for code-generating LLMs

Use this when you ask an LLM to write one file of the project. Paste the brief below first, then core/core.h, io/io.h and only the plan sections listed for the task. Do not paste the whole plan: unrelated sections dilute the result.

A.1 The brief (paste this first)

You are writing one file of a C17 project that benchmarks select, poll, epoll and io_uring servers on Linux. Follow these rules exactly.

1. core/core.h and io/io.h are fixed. Do not change, rename or redeclare anything in them. If you think something is missing, stop and say what, instead of working around it.

2. Write only the file(s) named in the task. Do not touch other files.

3. Use each mechanism the normal way. Add no optimizations. Every extra feature is a command-line flag already declared in core_config_t, off by default.

4. Apps never include epoll.h, poll.h or liburing.h. Modules never contain app logic.

5. No malloc or free inside the event loop. Use core_slot_alloc, core_slot_free and the connection table.

6. Callbacks run only from inside io_run(), never from inside an io_ call.

7. At most one read or write in flight per connection.

8. Update g_stats exactly as described in section D of io.h.

9. Never free a connection or a buffer while an async request on it is in flight.

10. Compile with: gcc -std=c17 -D_GNU_SOURCE -O2 -g -fno-omit-frame-pointer -Wall -Wextra, with no warnings.

11. If something is unclear, or marked CONFIRM or SETUP in the plan text you were given, state your assumption in a comment at the top of the file. Do not guess silently.

12. Output the complete file, then a list of every assumption you made and every part you are unsure about, especially buffer lifetimes, partial reads and writes, and completion handling.

A.2 What to give the LLM for each task

Task	Plan sections to paste	Also give
core/core.c (flag parser, connection table, pools, stats, signals, core_main)	4, 5, 6, 12.3	core.h, io.h
io/io_select.c	4, 5.1, 5.3, 10.1, 10.2	core.h, io.h
io/io_poll.c	4, 5.1, 5.3, 10.1, 10.3	core.h, io.h
io/io_epoll.c	4, 5.1 to 5.3, 6.3, 10.1, 10.4	core.h, io.h
io/io_uring.c	4, 5.1 to 5.3, 6.4, 10.1, 10.5, and the kernel version from section 14	core.h, io.h
apps/echo.c	4, 5.4, 7.1, 7.5	core.h, io.h
apps/filesrv.c	4, 5.2 to 5.4, 8.1, 8.5	core.h, io.h
tests (correctness gate)	13 and the owner section of the app being tested	core.h, io.h
tools/run_experiment	11, 12, 14	the run configuration format

Apache patches	9	The real Apache and APR source files for your exact versions. Paste the files; do not rely on a description

A.3 How to work with the output

- One file per request. Compile it, then run the correctness gate (section 13) before asking for the next file.
- Have someone else review each file against section D of io.h. Read io_uring and edge-triggered epoll code by hand: buffer lifetimes, partial writes, completion handling and stale completions are where generated code goes wrong.
- Keep a log of the assumptions each generated file lists, and resolve them in the plan or the headers.
- If a header needs to change, change it for everyone, update this plan, and regenerate only the files that depend on the change.

A.4 Settle these before generating

- Every **[CONFIRM]** item in section 15.2, or tell the LLM the default to assume.
- The kernel version on both machines, which decides which io_uring flags can be implemented.
- The httpd and APR versions, before any Apache work.
- The headers were written from this plan. They add one call the plan did not originally list, io_write_more, used to send an HTTP header with MSG_MORE ahead of sendfile or splice. Review them as a team before anyone builds on them.